

Guía de Pruebas de Rendimiento

JMeter para my-notes-js

Pruebas de carga sobre GitHub Pages · 50-100 usuarios concurrentes

Proyecto my-notes-js	Deploy GitHub Pages	Herramienta Apache JMeter	Versión guía Mayo 2026
--------------------------------	-------------------------------	-------------------------------------	----------------------------------

1. ¿Qué es JMeter y qué vamos a medir?

Apache JMeter es una herramienta de código abierto que simula múltiples usuarios haciendo peticiones HTTP a una aplicación web al mismo tiempo. Permite medir cómo responde el servidor bajo distintos niveles de carga.

En este ejercicio la app está desplegada en GitHub Pages. Vamos a medir:

Métrica	Qué nos dice
Tiempo de respuesta	Cuánto tarda el servidor en devolver index.html, CSS y JS
Throughput	Cuántas peticiones por segundo puede manejar el servidor
Tasa de error	Porcentaje de peticiones que devuelven error (4xx, 5xx)
Percentil 90 (P90)	El tiempo bajo el cual cae el 90% de las peticiones
Percentil 95 (P95)	Umbral de rendimiento más estricto — detecta picos extremos
Concurrencia	Comportamiento del servidor con 50 y 100 usuarios simultáneos

i Importante sobre GitHub Pages GitHub Pages es un servidor estático con CDN global. No tiene backend ni base de datos.

Los tiempos de respuesta serán muy buenos (< 200ms normalmente). El objetivo del ejercicio

es aprender a usar JMeter y leer métricas, no encontrar cuellos de botella reales.

2. Prerequisitos

Requisito	Detalle
Java 8+	JMeter corre sobre la JVM. Verificar con: <code>java -version</code>
JMeter ≥ 5.6	Descarga gratuita desde jmeter.apache.org
URL de la app	El docente entregará la URL de GitHub Pages activa
Conexión a internet	Para hacer peticiones reales al servidor de GitHub Pages

3. Instalación de JMeter

3.1 Descargar JMeter

1. **Ir a** https://jmeter.apache.org/download_jmeter.cgi
2. **Descargar** apache-jmeter-5.6.x.zip (o .tgz en Linux/Mac)
3. **Descomprimir** en una carpeta sin espacios en el path. Ej: C:\jmeter\ o ~/jmeter/

🚫 **Ruta sin espacios** Evitar rutas como C:\Program Files\jmeter\ — los espacios en el path causan errores al iniciar JMeter desde la línea de comandos.

3.2 Iniciar JMeter

```
# Mac / Linux
cd ~/jmeter/apache-jmeter-5.6.x/bin
./jmeter.sh

# Windows (doble clic o desde CMD)
cd C:\jmeter\apache-jmeter-5.6.x\bin
jmeter.bat
```

Se abrirá la interfaz gráfica (GUI). Para pruebas reales se usa modo non-GUI, pero durante el aprendizaje usaremos la GUI para ver la configuración visualmente.

4. Estructura del Test Plan

Un Test Plan en JMeter tiene una jerarquía de elementos. Esta es la estructura que vamos a construir:

```
Test Plan
├── Thread Group (define los usuarios y la carga)
│   ├── HTTP Request Defaults (URL base de la app)
│   ├── HTTP Request - Página principal (GET /)
│   ├── HTTP Request - Estilos (GET /css/styles.css)
│   ├── HTTP Request - JavaScript (GET /js/app.js)
│   ├── HTTP Header Manager (cabeceras HTTP comunes)
│   ├── Response Assertion (verificar que responde 200 OK)
│   ├── View Results Tree (listener: ver cada petición)
│   ├── Summary Report (listener: resumen estadístico)
│   └── Aggregate Report (listener: percentiles P90/P95)
```

Elemento	Para qué sirve
Thread Group	Representa los usuarios virtuales. Define cuántos y con qué cadencia.
HTTP Request Defaults	Configura la URL base una sola vez para todos los samplers.
HTTP Request	Una petición individual (sampler). Uno por recurso a probar.
HTTP Header Manager	Agrega cabeceras HTTP como Accept, User-Agent, etc.
Response Assertion	Valida que la respuesta sea correcta (código 200, texto esperado).
View Results Tree	Listener visual: muestra request/response de cada petición.
Summary Report	Listener: tabla con promedio, mín, máx, throughput, errores.
Aggregate Report	Listener: igual que Summary pero agrega percentiles P90 y P95.

5. Crear el Test Plan paso a paso

5.1 Nuevo Test Plan

4. **Abrir JMeter** → File > New o iniciar JMeter desde cero.
5. **Renombrar el Test Plan** → Click en 'Test Plan' en el árbol izquierdo > cambiar el nombre a 'Post-it Notes — Carga Intermedia'.
6. **Guardar** → File > Save As > nombre: postit-load-test.jmx

5.2 Configurar el Thread Group

7. **Click derecho en Test Plan** → Add > Threads (Users) > Thread Group.
8. **Configurar los parámetros de carga:**

Parámetro	Valor	Descripción
Number of Threads	50	Usuarios concurrentes en la primera prueba
Ramp-Up Period	30	Segundos para llegar a los 50 usuarios (no todos a la vez)
Loop Count	3	Cada usuario repite el ciclo 3 veces
Duration	—	Dejar vacío si usas Loop Count

💡 **¿Qué es el Ramp-Up?** Con Ramp-Up = 30s y 50 usuarios, JMeter agrega ~1.7 usuarios por segundo.

Esto simula una llegada gradual de usuarios, más realista que lanzarlos todos a la vez. Para la prueba de 100 usuarios, cambia Number of Threads a 100 y Ramp-Up a 60.

5.3 HTTP Request Defaults

9. Click derecho en **Thread Group** → Add > Config Element > HTTP Request Defaults.
10. Completar los campos:

Campo	Valor
Protocol	https
Server Name	rueeen.github.io ← la URL que entregará el docente
Port	443 (o dejar vacío para HTTPS)
Path	/my-notes-js

5.4 HTTP Header Manager

11. Click derecho en **Thread Group** → Add > Config Element > HTTP Header Manager.
12. Agregar las siguientes cabeceras:

Header	Valor
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language	es-CL,es;q=0.9,en;q=0.8
Accept-Encoding	gzip, deflate, br
Connection	keep-alive

5.5 HTTP Requests (Samplers)

Agrega una petición HTTP por cada recurso de la app. Click derecho en **Thread Group** → Add > Sampler > HTTP Request:

Sampler	Método	Path
Página principal	GET	/my-notes-js/ (o solo / si usaste el path en Defaults)
Estilos CSS	GET	/my-notes-js/css/styles.css
JavaScript	GET	/my-notes-js/js/app.js
Fuente Caveat	GET	/css2?family=Caveat:wght@600;700 (Google Fonts)

Sampler	Método	Path
Bootstrap CSS	GET	/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css
Bootstrap JS	GET	/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js

 **Recursos externos (CDN)** Bootstrap y Google Fonts se cargan desde CDNs externos (cdn.jsdelivr.net, fonts.googleapis.com).

Para probarlos, crea HTTP Requests separados con su propio Server Name.

En la práctica pueden omitirse si el foco es el servidor de GitHub Pages.

5.6 Response Assertion

13. **Click derecho en el HTTP Request de la página principal** → Add > Assertions > Response Assertion.

14. **Configurar:**

Campo	Valor
Field to Test	Response Code
Pattern Matching	Equals
Patterns to Test	200

Esto hace que JMeter marque la petición como FAILED si no recibe un HTTP 200 OK.

5.7 Agregar Listeners (reportes)

Click derecho en Thread Group → Add > Listener → agrega los tres siguientes:

Listener	Para qué usarlo
View Results Tree	Ver cada petición individual. Útil para debug. Desactivar en pruebas grandes.
Summary Report	Tabla con promedio, mín, máx, throughput y % error por sampler.
Aggregate Report	Igual que Summary pero incluye percentiles P90, P95 y P99.

 **View Results Tree consume memoria** Con 100 usuarios y 3 loops, JMeter puede guardar 1800+ resultados en memoria.

Desactivar (click derecho > Disable) antes de correr pruebas de carga reales.

Usarlo solo para verificar que las peticiones están bien configuradas.

6. Escenarios de prueba

Ejecuta los escenarios en orden. Cada uno tiene un objetivo diferente:

Escenario 1 — Línea base (10 usuarios)

Objetivo: establecer los tiempos de respuesta sin carga para comparar.

Parámetro	Valor
Number of Threads	10
Ramp-Up Period	10 segundos
Loop Count	3
Qué observar	Tiempo promedio, máximo y P90 con carga mínima

Escenario 2 — Carga intermedia (50 usuarios)

Objetivo: simular uso normal de la app con varios estudiantes conectados.

Parámetro	Valor
Number of Threads	50
Ramp-Up Period	30 segundos
Loop Count	3
Qué observar	Comparar P90 con escenario 1. ¿Sube el tiempo de respuesta?

Escenario 3 — Carga alta (100 usuarios)

Objetivo: ver el comportamiento del servidor bajo presión sostenida.

Parámetro	Valor
Number of Threads	100
Ramp-Up Period	60 segundos
Loop Count	3
Qué observar	Aumento de errores, saturación, degradación del P95

📌 **Ejercicio comparativo** Completa los tres escenarios y compara los valores de P90 en el Aggregate Report.

¿Se degrada el rendimiento al pasar de 10 a 100 usuarios?

¿La tasa de error se mantiene en 0% en todos los escenarios?

7. Ejecutar la prueba

7.1 Desde la interfaz gráfica (GUI)

15. **Verificar configuración** → revisar que la URL en HTTP Request Defaults apunta a la URL correcta.
16. **Limpiar resultados anteriores** → Run > Clear All (o Ctrl+Shift+E).
17. **Iniciar la prueba** → Run > Start (o Ctrl+R). El contador de hilos activos aparece arriba a la derecha.
18. **Observar en tiempo real** → click en Summary Report para ver los resultados mientras corre.
19. **Detener si es necesario** → Run > Stop (Ctrl+.) para parada inmediata, o Run > Shutdown para esperar que terminen los hilos activos.

7.2 Desde línea de comandos (modo non-GUI)

El modo non-GUI consume mucha menos memoria y es el recomendado para pruebas reales:

```
# Correr el test y guardar resultados en CSV
jmeter -n -t postit-load-test.jmx -l resultados.csv -e -o reporte/

# Parámetros:
#   -n          modo non-GUI
#   -t          archivo .jmx del test plan
#   -l          archivo de salida con resultados
#   -e          generar reporte HTML al terminar
#   -o          carpeta donde guardar el reporte HTML

# Abrir el reporte HTML generado:
open reporte/index.html      # Mac
start reporte/index.html     # Windows
```

8. Interpretar los resultados

8.1 Métricas del Aggregate Report

Métrica	Cómo interpretar
# Samples	Total de peticiones enviadas en toda la prueba.
Average (ms)	Tiempo de respuesta promedio. Meta: < 500ms para sitios estáticos.
Min / Max (ms)	Tiempo mínimo y máximo. Un Max muy alto indica picos de latencia.
P90 (ms)	El 90% de las peticiones respondió en este tiempo o menos. Indicador principal.
P95 (ms)	Umbral más estricto. Si P95 >> P90 hay peticiones lentas puntuales.
Throughput	Peticiones por segundo que el servidor procesó. Mayor es mejor.
Error %	Porcentaje de peticiones fallidas. Meta: 0% en sitios estáticos.
Received KB/s	Datos recibidos por segundo. Útil para detectar cuellos de banda.

8.2 Umbrales de referencia para sitios estáticos

Tiempo de respuesta	Calificación	Impacto en el usuario
< 200ms	Excelente	Respuesta casi instantánea para el usuario.
200-500ms	Bueno	Aceptable para la mayoría de casos de uso.
500ms-1s	Aceptable	El usuario puede notar la demora.
> 1s	Malo	Degradación notable. Investigar causa.
> 3s	Crítico	Alta probabilidad de abandono del usuario.

8.3 Tabla de comparación entre escenarios

Usa esta tabla para registrar y comparar los resultados de los tres escenarios:

Escenario	Promedio (ms)	P90 (ms)	P95 (ms)	Error %
10 usuarios	—	—	—	—
50 usuarios	—	—	—	—
100 usuarios	—	—	—	—

9. Problemas comunes y soluciones

Problema	Solución
Connection refused	Verificar que la URL de GitHub Pages esté activa y accesible desde el navegador.
SSL handshake error	En HTTP Request Defaults, marcar la opción 'Use KeepAlive' y verificar que el protocolo sea https.
Error 429 Too Many Requests	GitHub Pages puede limitar peticiones excesivas. Reducir usuarios o agregar Timer de 1-2s entre peticiones.
OutOfMemoryError en JMeter	Desactivar View Results Tree. Si persiste: aumentar heap en jmeter.bat/-sh: -Xmx1g a -Xmx2g.
Tiempos inconsistentes	Las primeras peticiones son más lentas (cold start del CDN). Agregar un 'Warm-up' de 5 usuarios antes del grupo principal.
Throughput muy bajo	Agregar Constant Timer (1000ms) entre requests para simular tiempo de lectura del usuario real.
Error % > 0%	Revisar View Results Tree para ver qué código HTTP devuelve. Verificar el path en cada HTTP Request.

10. Resumen: checklist del ejercicio

✓ **Configuración mínima necesaria** Thread Group con Number of Threads, Ramp-Up y Loop Count

HTTP Request Defaults con la URL base de GitHub Pages

Al menos 3 HTTP Requests: index.html, styles.css, app.js

Response Assertion verificando código 200

Summary Report y Aggregate Report como listeners

📄 **Entregable sugerido** Capturas de pantalla del Aggregate Report para los 3 escenarios

Tabla comparativa con los valores de P90, P95 y Error % completados

Conclusión: ¿Se degrada el rendimiento? ¿En qué escenario y por qué?

🎯 **Meta de rendimiento esperada para GitHub Pages** P90 < 300ms en todos los escenarios (es un servidor CDN estático)

Error % = 0% en escenarios 1 y 2, posible > 0% en escenario 3 por rate limiting

Throughput estable o con caída < 20% al pasar de 10 a 100 usuarios

github.com/rueeen/my-notes-js · Guía generada Mayo 2026